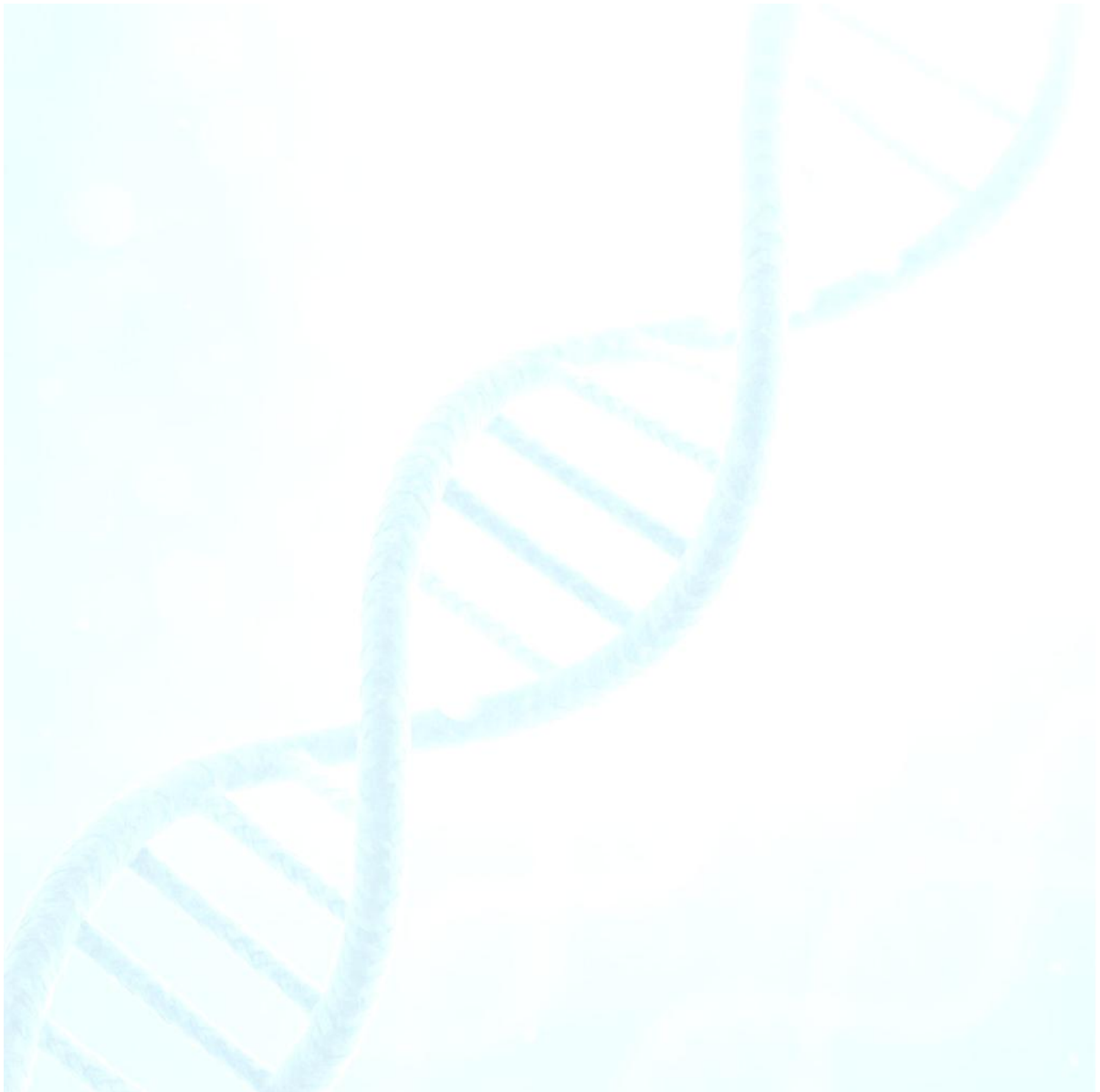


Guide de Traduction Dynamique (FR / EN)

Stage de Lorenzo Lienard, 1^{er} Année BTS informatique Lycée Wallon



Objectif

Ce document explique pas à pas comment rendre le site totalement bilingue en exploitant la structure HTML actuelle (les boutons de changement de langue) et en injectant la logique avec JavaScript natif, sans recharger la page.

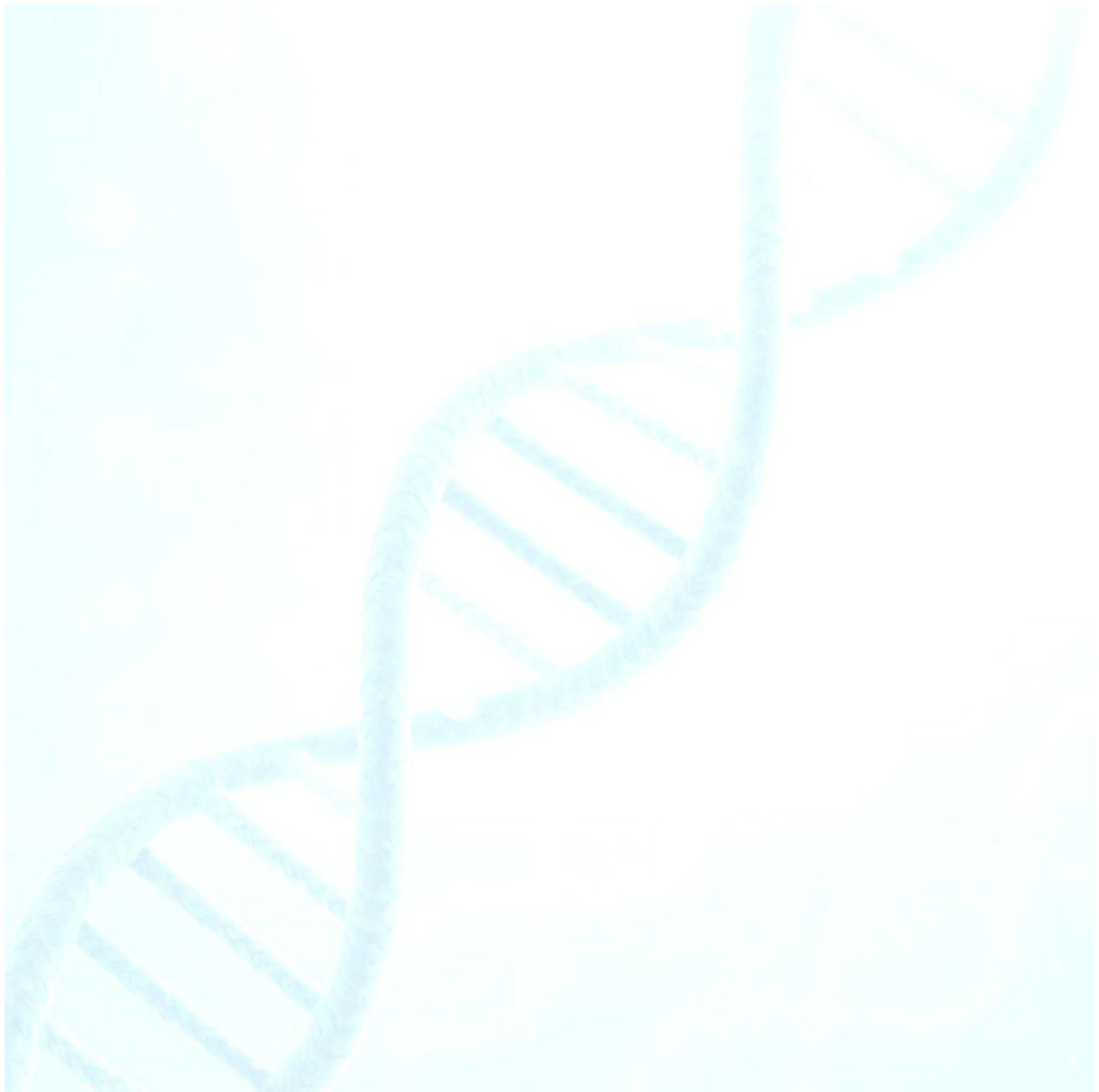
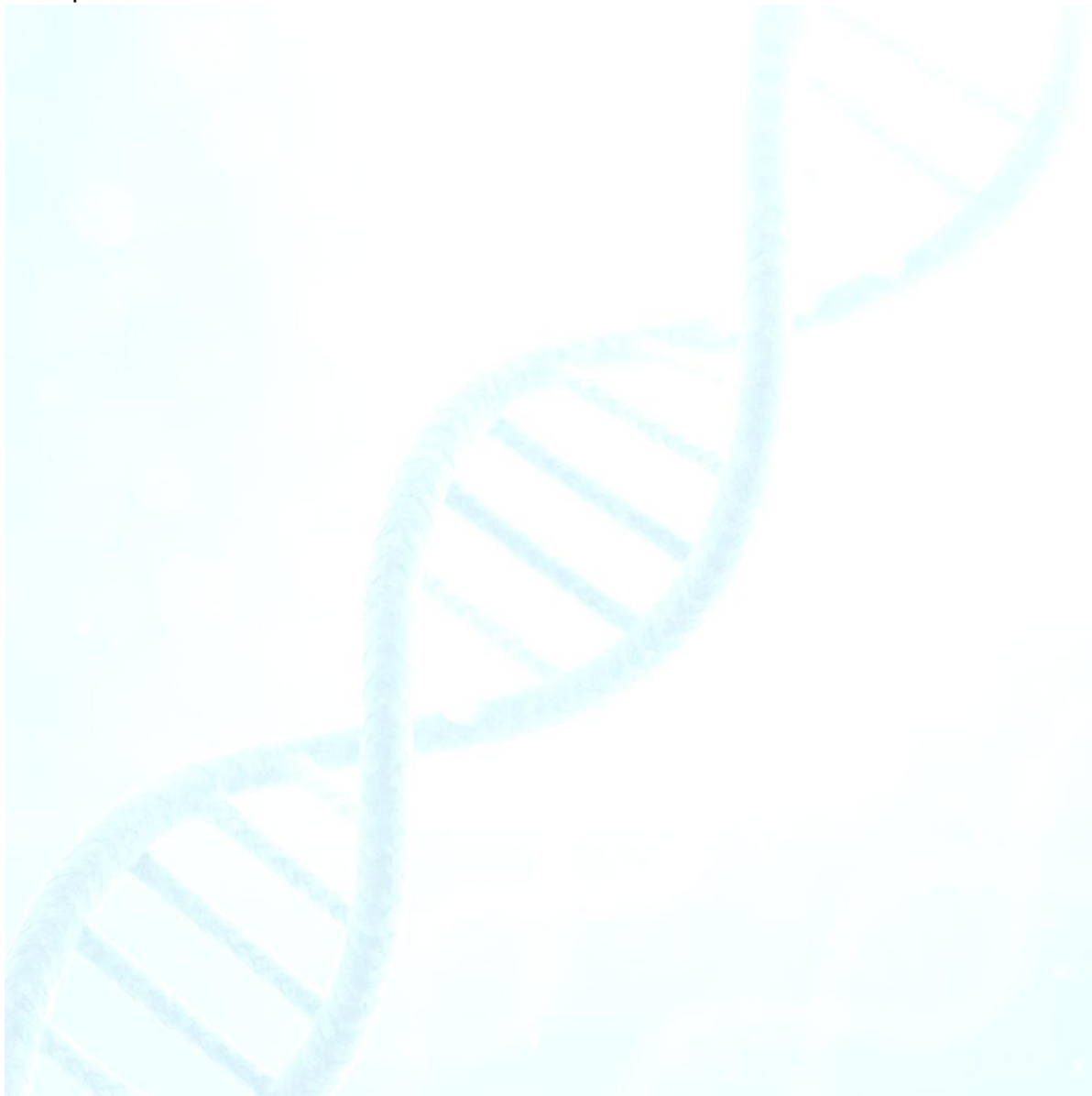


Table des matières

Objectif	2
1. Principe de fonctionnement	4
2. Étape 1 : Préparer le fichier HTML.....	5
3. Étape 2 : Inclure le script dans le HTML	7
Pourquoi cette méthode ?	7

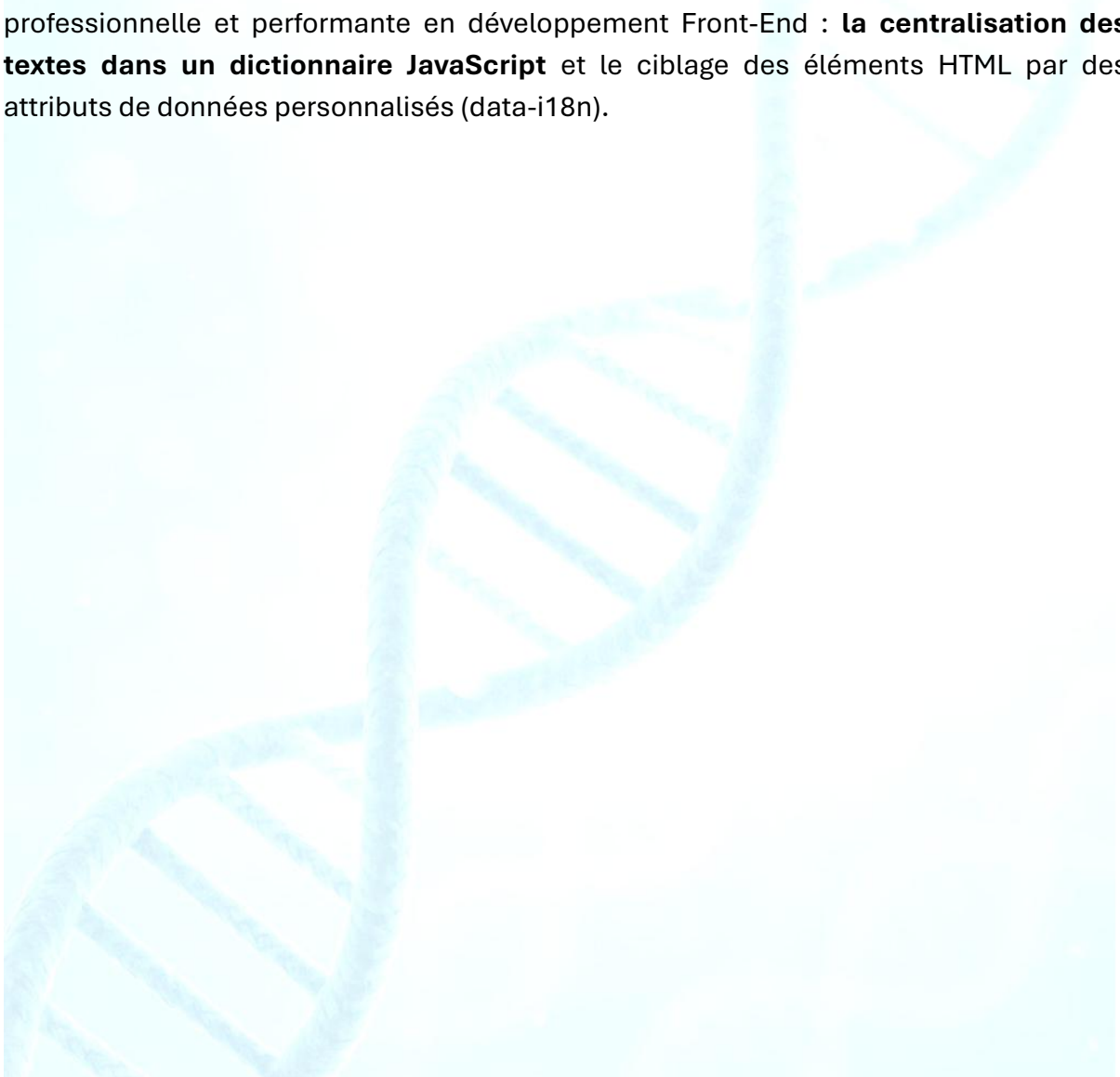


1. Principe de fonctionnement

Dans le code actuel de index.html, deux boutons sont déjà présents dans le header :

```
<div class="language-toggle">
  <button class="lang-btn active">FR</button>
  <button class="lang-btn">EN</button>
</div>
```

Pour lier ces boutons à une traduction instantanée, nous allons appliquer une méthode professionnelle et performante en développement Front-End : **la centralisation des textes dans un dictionnaire JavaScript** et le ciblage des éléments HTML par des attributs de données personnalisés (data-i18n).



2. Étape 1 : Préparer le fichier HTML

Plutôt que de dupliquer les pages ou de créer un deuxième fichier HTML, il faut ajouter un attribut `data-il8n` sur chaque élément de texte qui doit changer de langue. La valeur de cet attribut servira de "clé" de traduction.

Exemple créer dans le dossier un fichier nommé `scripts/translate.js` :

```
const translations = {
  fr: {
    "nav-philosophy": "Philosophie",
    "nav-modules": "Modules",
    "nav-clinstore": "ClinStore",
    "nav-sdk": "SDK",
    ..... a toi de faire la suite 😊
  },
  en: {
    "nav-philosophy": "Philosophy",
    "nav-modules": "Modules",
    "nav-clinstore": "ClinStore",
    "nav-sdk": "SDK",
    ..... et la suite en anglais 😊
  }
};

// Fonction pour appliquer la langue choisie
function setLanguage(lang) {
  // 1. Rechercher tous les éléments possédant l'attribut data-il8n
  const elements = document.querySelectorAll('[data-il8n]');

  elements.forEach(element => {
    const key = element.getAttribute('data-il8n');
    if (translations[lang] && translations[lang][key]) {
      // Utilisation de innerHTML au cas où il y aurait des balises
      // comme <br> ou <span>
      element.innerHTML = translations[lang][key];
    }
  });

  // 2. Mettre à jour l'attribut lang de la balise <html> pour le SEO
  document.documentElement.lang = lang;
  // 3. Sauvegarder le choix pour les prochaines visites
  localStorage.setItem('preferred-lang', lang);
}

// Écoute des clics sur l'interface
document.addEventListener('DOMContentLoaded', () => {
  const buttons = document.querySelectorAll('.language-toggle .lang-
  btn');
```

```
buttons.forEach(button => {
  button.addEventListener('click', (e) => {
    // Supprimer la classe active de tous les boutons
    buttons.forEach(btn => btn.classList.remove('active'));

    // Ajouter la classe active sur le bouton cliqué
    e.target.classList.add('active');

    // Récupérer la langue choisie (basée sur le texte FR ou EN)
    const chosenLang = e.target.textContent.trim().toLowerCase();
    setLanguage(chosenLang);
  });
});

// Charger la langue préférée au démarrage (par défaut 'fr')
const savedLang = localStorage.getItem('preferred-lang') || 'fr';
setLanguage(savedLang);

// Ajuster l'état visuel des boutons au chargement initial
buttons.forEach(btn => {
  if (btn.textContent.trim().toLowerCase() === savedLang) {
    btn.classList.add('active');
  } else {
    btn.classList.remove('active');
  }
});
});
```



3. Étape 2 : Inclure le script dans le HTML

À la toute fin du fichier index.html, juste au-dessus de la fermeture de la balise `</body>`, il faut appeler ce nouveau script. Il faut donc placer juste avant le script de téléchargement actuel :

```
<script src="scripts/translate.js"></script>
```

Ensuite pour chaque composant HTML comportant du texte, on appelle la clé `data-i18n` avec la valeur de traduction définie dans le script pour ce composant

Exemple :

```
<h2 data-i18n="philosophy-title">Notre Philosophie</h2>
```

Si dans le script on a défini :

```
fr: {  
  // ... ses clés existantes  
  "philosophy-title": "Notre Philosophie"  
},  
en: {  
  // ... ses clés existantes  
  "philosophy-title": "Our Philosophy"  
}
```

Pourquoi cette méthode ?

Cette technique montre la manipulation du DOM (Document Object Model) en Vanilla JavaScript pur, l'utilisation de l'API `LocalStorage` pour la persistance des choix de l'utilisateur, et l'usage sémantique des attributs de données HTML5. C'est propre, moderne et exempt d'effets de bord.